

文献翻译

题 目 Atmosphere: 基于 Rust 和 Verus 的
实用型已验证内核

学生姓名 苏爽

专业班级 计算机科学与技术(嵌入式软件)22-04

学 号 542213430420

学 院 计算机与人工智能学院

指导教师(职称) 张平原（讲师）

完成时间 2026 年 3 月 15 日

Atmosphere: 基于 Rust 和 Verus 的实用型已验证内核

Xiangdong Chen, Zhaofeng Li, Jerry Zhang, Vikram Narayanan, Anton Burtsev

University of Utah, Palo Alto Networks

SOSP '25: Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles

Pages 752 - 767

<https://doi.org/10.1145/3731569.3764821>

摘要: 近年来, 程序设计语言和自动化形式推理的发展, 正在改变形式化验证系统开发中复杂性与实用性之间的关系。本文利用 Verus 开发了一个经过形式化验证的微内核 Atmosphere。Verus 是一种面向 Rust 的新型验证工具, 它结合了线性类型、权限化推理以及基于 SMT 的自动化验证方法, Atmosphere 是一个功能较完整的微内核, 支持混合关键性系统中的严格隔离。本文全部代码均使用 Rust 编写, 并使用 Verus 证明其功能正确性, 即证明具体实现是高层规格的精化。Atmosphere 包含约 6000 行可执行代码, 其开发和验证总共耗费不到 2.5 人年, 其中约 1.5 人年用于验证工作。平均来看, 其证明代码与执行代码比例约为 3.32:1, 并且在现代笔记本电脑上完成验证所需时间少于 20 秒。作者认为, 这说明基于 Verus 的已验证系统开发具有较强的实践可行性。

关键词: 操作系统; 微内核; 形式化验证; Rust; Verus

1. 引言

长期以来, 系统软件的复杂性使其难以被形式化验证。虽然已有多个项目展示了已验证内核开发的可能性, 但这类工作通常需要较强的形式化专业知识和大量人工证明工作。例如, 最早完成形式化验证的微内核 seL4, 其开发和证明工作耗费了大量时间, 其中不仅包括内核本身的证明, 还包括形式化语言框架、证明工具和库的开发。

为了降低验证复杂度, 已有研究尝试开发具有高度证明自动化能力的程序语言和工具, 也有工作探索将内核代码直接转换为 SMT 表达式以实现“按钮式验证”。这些方法虽然在自动化方面取得一定效果, 但往往需要对内核组织方式和接口做出简化假设, 因此难以完全适用于复杂、功能丰富的真实内核。

本文认为, 自动化验证工具已经可以用于底层系统的开发与验证。具体来说, 本文展示了如何使用 Verus 这一面向 Rust 的 SMT 验证工具, 开发一个完全经过验证的微内核 Atmosphere。Atmosphere 与经典微内核类似, 支持多处理器硬件、进程和线程、动态内存管理、进程间通信、虚拟地址空间、IOMMU 以及容器抽象。Atmosphere 被设计为一个分离内核, 用于支持混合关键性系统中的严格隔离。

为了使语义复杂、以指针为中心的内核代码能够被实际验证, 本文采用了几个关键设计选择。第一是以指针为中心的设计。Rust 的线性类型系统并不天然适合表达复杂

的非线性数据结构，因此 Atmosphere 主动使用原始指针来构建内核数据结构，但通过 Verus 证明所有指针操作的正确性。第二是扁平化权限存储。系统将复杂层级结构中对象的访问权限统一保存在子系统顶层的扁平映射中，从而避免大量递归证明。第三是手动内存管理。Atmosphere 不依赖 Rust 自动内存管理来证明内核对象生命周期，而是显式分配和释放内存，并通过证明保证内存操作安全和无泄漏。

2. Verus 背景

Verus 是一种面向 Rust 程序的形式化验证框架。与通用定理证明器不同，Verus 牺牲了一部分通用性，以换取更高度度的自动化。它主要用于证明 Rust 程序，特别是底层系统程序的功能正确性。Verus 会将 Rust 程序转换为 SMT 表达式，并交由 Z3 等求解器处理。

为了提高验证速度，Verus 利用 Rust 的类型系统，尤其是线性类型系统，来辅助处理别名和内存问题，而不是完全依赖 SMT 求解器。这样可以大幅降低 SMT 查询的复杂度，提高验证的速度和可扩展性。对于双向链表、树结构和原始指针等非线性数据结构，Verus 借鉴了基于权限的分离逻辑思想，使开发者能够对内核中常见的复杂数据结构进行推理。

Verus 允许开发者在同一种 Rust 风格语言中书写可执行代码、规格和证明。通常的验证流程是：先定义系统的抽象状态和接口预期行为，再证明具体实现是高层抽象规格的精化。Verus 中的 `ghost` 变量可用于表示抽象系统状态，也可辅助证明。规格和证明代码只在验证阶段存在，编译时会被擦除，因此不会引入运行时开销。

3. 系统架构

Atmosphere 是一个功能较完整的微内核，支持地址空间、线程、内存管理、中断分发、进程间通信、IOMMU 和容器等机制。设备驱动、网络栈、文件系统等服务运行在用户态进程中。为了简化验证，Atmosphere 在多处理器环境下采用大锁机制，即所有中断和系统调用都在一个全局锁保护下执行。

Atmosphere 中的进程可以通过端点进行通信。发送线程可以传递标量数据、内存页引用、IOMMU 标识符以及其他端点引用。端点使进程之间能够建立共享内存区域，从而支持高效的异步通信。

在隔离方面，Atmosphere 使用容器抽象来支持进程组之间的可证明隔离。容器拥有确定的内存和 CPU 资源配额，父容器可以将自身资源的一部分分配给子容器。系统

调用规格被设计为支持用户态程序的功能正确性证明和容器之间的非干扰证明。

4. 验证方法

为了证明 Atmosphere 正确运行，本文主要证明两个性质：精化和良构性。精化定理证明微内核具体实现是其高层抽象规格的精化，即实现满足功能正确性。良构性定理证明每次状态转换后，系统状态仍保持相应的不变量。

本文将微内核建模为一个作用在抽象状态上的状态机。每一次内核调用都会依据高层规格更新抽象状态，同时系统维护一组不变量，用于描述各类内核数据结构的正确性以及安全性、隔离性等跨模块性质。

Atmosphere 采用扁平化、指针中心的设计。对于递归指针数据结构，系统允许使用类似 C 语言的原始指针，但对象访问权限不嵌入递归结构内部，而是统一保存在子系统顶层的扁平映射中。这样可以将许多递归规格改写为非递归规格，并将结构性证明和非结构性证明分离开来，从而降低 SMT 验证难度。

5. 实验与验证规模

Atmosphere 的微内核实现包含约 6000 行可执行代码，同时包含约 20100 行证明相关代码，其中约 14300 行为规格代码，约 5800 行为验证提示和证明代码。在规格代码中，约 2900 行用于描述内核系统调用接口的顶层抽象规格。

从规模上看，Atmosphere 的证明代码与可执行代码比例约为 3.32:1，低于一些早期验证系统。更重要的是，通过谨慎的系统设计，Atmosphere 能够在现代笔记本电脑上于 20 秒内完成验证。这个时间甚至短于编译内核本身所需时间，因此可以支持较为交互式的开发过程，而不是传统的“重新编译、重启、运行测试”的开发方式。

6. 评估

本文从验证复杂度、扁平化设计影响、开发速度、微基准测试、设备驱动和应用基准等方面对 Atmosphere 进行了评估。实验结果表明，扁平化权限存储可以有效降低递归数据结构证明的复杂性，使验证更容易扩展到内核中的树、链表等复杂结构。

在性能方面，Atmosphere 的用户态设备驱动和应用程序能够达到较高的运行效率。对于网络和存储相关实验，Atmosphere 在部分场景下接近或达到高性能用户态驱动框架的性能水平。作者认为，这说明经过验证的微内核并不必然牺牲实用性能。

7. 相关工作

早期操作系统验证工作主要追求高安全等级保证，但受限于当时工具能力，进展有限。seL4 是第一个展示实用微内核形式化验证可行性的系统，其使用 Isabelle/HOL 对 C 实现进行了大规模证明。CertiKOS 等项目则使用 Coq 对并发内核进行验证。与这些工作相比，Atmosphere 同样关注高性能内核结构，但更强调利用 Verus 提高自动化程度，降低证明成本。

此外，Hyperkernel、IronFleet、Dafny、Ivy 等研究也探索了不同形式的系统软件验证方法。Verus 与这些方法相比，更适合在 Rust 语言环境中同时表达执行代码、规格和证明，并借助 Rust 类型系统与 SMT 求解器共同完成验证。

8. 结论

本文展示了使用 Rust 和 Verus 开发实用型已验证微内核的可行性。Atmosphere 通过指针中心设计、扁平化权限存储和手动内存管理等关键设计，在保留底层系统代码性能优势的同时，实现了较高级别的形式化验证。实验结果表明，Verus 的自动化能力可以显著降低已验证系统开发的成本，使形式化验证更接近实际系统开发流程。

该工作说明，Rust 语言和 Verus 工具的结合，为构建高可信系统软件提供了新的技术路线。对于后续微内核、访问控制模块以及能力系统的验证研究而言，Atmosphere 提供了重要参考。

参考文献

- [1] Gerwin Klein, Kevin Elphinstone, Gernot Heiser, June Andronick, David Cock, Philip Derrin, Dhammika Elkaduwe, Kai Engelhardt, Rafal Kolanski, Michael Norrish, Thomas Sewell, Harvey Tuch, and Simon Winwood. seL4: Formal verification of an OS kernel. In Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles (SOSP), October 11, 2009.
- [2] Ronghui Gu, Zhong Shao, Hao Chen, Xiongnan (Newman) Wu, Jieung Kim, Vilhelm Sjöberg, and David Costanzo. CertiKOS: An extensible architecture for building certified concurrent OS kernels. In 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI), 2016.
- [3] Jean Yang and Chris Hawblitzel. Safe to the last instruction: Automated verification of a type-safe operating system. In Proceedings of the 31st ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI), 2010.
- [4] Chris Hawblitzel, Jon Howell, Jacob R. Lorch, Arjun Narayan, Bryan Parno, Danfeng Zhang, and Brian Zill. Ironclad Apps: End-to-end security via automated full-system verification. In 11th USENIX Symposium on Operating Systems Design and Implementation (OSDI), October 2014.
- [5] Chris Hawblitzel, Jon Howell, Manos Kapritsos, Jacob R. Lorch, Bryan Parno, Michael L. Roberts, Srinath Setty, and Brian Zill. IronFleet: Proving practical distributed systems correct. In Proceedings of the 25th Symposium on Operating Systems Principles (SOSP), 2015.
- [6] K. Rustan M. Leino. Dafny: An automatic program verifier for functional correctness. In Logic for Programming, Artificial Intelligence, and Reasoning, 2010.
- [7] Luke Nelson, Helgi Sigurbjarnarson, Kaiyuan Zhang, Dylan Johnson, James Bornholt, Emina Torlak, and Xi Wang. Hyperkernel: Push-button verification of an OS kernel. In Proceedings of the 26th Symposium on Operating Systems Principles (SOSP), October 14, 2017.
- [8] Helgi Sigurbjarnarson, James Bornholt, Emina Torlak, and Xi Wang. Push-button verification of file systems via crash refinement. In Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI), November 2016.
- [9] Luke Nelson, James Bornholt, Ronghui Gu, Andrew Baumann, Emina Torlak, and Xi Wang. Scaling symbolic evaluation for automated verification of systems code with Serval. In Proceedings of

- f the 27th ACM Symposium on Operating Systems Principles (SOSP), 2019.
- [10] Oded Padon, Kenneth L. McMillan, Aurojit Panda, Mooly Sagiv, and Sharon Shoham. Ivy: Safety verification by interactive generalization. In Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI), 2016.
 - [11] Helgi Sigurbjarnarson, Luke Nelson, Bruno Castro-Karney, James Bornholt, Emina Torlak, and Xi Wang. Nickel: A framework for design and verification of information flow control systems. In Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI), October 2018.
 - [12] Can Cebeci, Yonghao Zou, Diyu Zhou, George Candea, and Clément Pit-Claudel. Practical verification of system-software components written in standard C. In Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles (SOSP), 2024.
 - [13] Philip Wadler. Linear types can change the world! In Proceedings of the IFIP Working Group 2.2, 2.3 Working Conference on Programming Concepts and Methods, number 4, 1990.
 - [14] Andrea Lattuada, Travis Hance, Chanhee Cho, Matthias Brun, Isitha Subasinghe, Yi Zhou, Jon Howell, Bryan Parno, and Chris Hawblitzel. Verus: Verifying Rust programs using linear ghost types. OOPSLA1, April 6, 2023.
 - [15] Jialin Li, Andrea Lattuada, Yi Zhou, Jonathan Cameron, Jon Howell, Bryan Parno, and Chris Hawblitzel. Linear types for large-scale systems verification. Proceedings of the ACM on Programming Languages, OOPSLA1, April 2022.
 - [16] Vytautas Astrauskas, Peter Müller, Federico Poli, and Alexander J. Summers. Leveraging Rust types for modular specification and verification. Proceedings of the ACM on Programming Languages, OOPSLA, October 2019.
 - [17] Richard Bornat, Cristiano Calcagno, Peter O’Hearn, and Matthew Parkinson. Permission accounting in separation logic. In Proceedings of the 32nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL), 2005.
 - [18] Peter Müller, Malte Schwerhoff, and Alexander J. Summers. Viper: A verification infrastructure for permission-based reasoning. In Verification, Model Checking, and Abstract Interpretation, 2016.
 - [19] Vytautas Astrauskas, Aurel Bily, Jonáš Fiala, Zachary Grannan, Christoph Matheja, Peter Müller, Federico Poli, and Alexander J. Summers. The Prusti Project: Formal verification for Rust. In N

- ASA Formal Methods, 2022.
- [20] Ralf Jung, Jacques-Henri Jourdan, Robbert Krebbers, and Derek Dreyer. RustBelt: Securing the foundations of the Rust programming language. *Proceedings of the ACM on Programming Languages*, POPL, December 27, 2017.
- [21] Gernot Heiser. Benchmarking crimes meet formal verification, April 27, 2025. URL: <https://microkerneldude.org/2025/04/27/benchmarking-crimes-meet-formal-verification/>.
- [22] Verus-Lang. Recursive exec and proof functions, proofs by induction. URL: <https://verus-lang.github.io/verus/guide/induction.html>.
- [23] Amit Levy, Bradford Campbell, Branden Ghena, Daniel B. Giffin, Pat Pannuto, Prabal Dutta, and Philip Levis. Multiprogramming a 64kB computer safely and efficiently. In *Proceedings of the 26th Symposium on Operating Systems Principles (SOSP)*, 2017.
- [24] Ziqiao Zhou, Anjali, Weiteng Chen, Sishuai Gong, Chris Hawblitzel, and Weidong Cui. VeriSMo: A verified security module for Confidential VMs. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.
- [25] Matthias Brun, Reto Achermann, Tej Chajed, Jon Howell, Gerd Zellweger, and Andrea Lattuada. Beyond isolation: OS verification as a foundation for correct applications. In *Proceedings of the 19th Workshop on Hot Topics in Operating Systems (HOTOS)*, June 22, 2023.
- [26] Verus-Lang. A verified doubly-linked list example in Verus. URL: https://github.com/verus-lang/verus/blob/main/examples/doubly_linked.rs.
- [27] The Rust Project. Reference cycles can leak memory. URL: <https://doc.rust-lang.org/book/ch15-06-reference-cycles.html>.
- [28] The Coq proof assistant. URL: <https://coq.inria.fr/>.
- [29] Tobias Nipkow, Markus Wenzel, and Lawrence C. Paulson. Isabelle/HOL: A Proof Assistant for Higher-Order Logic. *Lecture Notes in Computer Science*, 2002.
- [30] Leonardo de Moura, Soonho Kong, Jeremy Avigad, Floris van Doorn, and Jakob von Raumer. The Lean theorem prover. In *Automated Deduction - CADE-25*, 2015.
- [31] Andrea Lattuada, Travis Hance, Jay Bosamiya, Matthias Brun, Chanhee Cho, Hayley LeBlanc, Pranav Srinivasan, Reto Achermann, Tej Chajed, Chris Hawblitzel, Jon Howell, Jacob R. Lorch, Oded Padon, and Bryan Parno. Verus: A practical foundation for systems verification. In *Proceedings of the 30th Symposium on Operating Systems Principles (SOSP)*. ACM, November 2024.

- [32] Nikhil Swamy, Cătălin Hrițcu, Chantal Keller, Aseem Rastogi, Antoine Delignat-Lavaud, Simon Forest, Karthikeyan Bhargavan, Cédric Fournet, Pierre-Yves Strub, Markulf Kohlweiss, Jean-Karim Zinzindohoue, and Santiago Zanella-Béguelin. Dependent types and multi-monadic effects in F*. In Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL), 2016.
- [33] Mike Barnett, Bor-Yuh Evan Chang, Robert DeLine, Bart Jacobs, and K. Rustan M. Leino. Boogie: A modular reusable verifier for object-oriented programs. In Proceedings of the 4th International Conference on Formal Methods for Components and Objects (FMCO), 2005.
- [34] Leonardo de Moura and Nikolaj Bjørner. Z3: An efficient SMT solver. In International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS), Lecture Notes in Computer Science, 2008.
- [35] Liam O’Connor, Zilin Chen, Christine Rizkallah, Vincent Jackson, Sidney Amani, Gerwin Klein, Toby Murray, Thomas Sewell, and Gabriele Keller. Cogent: Uniqueness types and certifying compilation. *Journal of Functional Programming*, 2021.
- [36] Xavier Denis, Jacques-Henri Jourdan, and Claude Marché. Creusot: A foundry for the deductive verification of Rust programs. In *Formal Methods and Software Engineering*, 2022.
- [37] Son Ho and Jonathan Protzenko. Aeneas: Rust verification by functional translation. *Proceedings of the ACM on Programming Languages*, ICFP, August 2022.
- [38] John Boyland. Checking interference with fractional permissions. In *Static Analysis*, 2003.
- [39] Paul Emmerich, Maximilian Pudelko, Simon Bauer, and Georg Carle. User space network drivers. In *Proceedings of the Applied Networking Research Workshop (ANRW)*, 2018.
- [40] Vikram Narayanan, Tianjiao Huang, David Detweiler, Dan Appel, Zhaofeng Li, Gerd Zellweger, and Anton Burtsev. RedLeaf: Isolation and communication in a safe Operating System. In *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2020.
- [41] Aurojit Panda, Sangjin Han, Keon Jang, Melvin Walls, Sylvia Ratnasamy, and Scott Shenker. NetBricks: Taking the V out of NFV. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, November 2016.
- [42] Keir Fraser, Steven Hand, Rolf Neugebauer, Ian Pratt, Andrew Warfield, Mark Williamson, et al. Safe hardware access with the Xen virtual machine monitor. In *1st Workshop on Operating Syst*

- em and Architectural Support for the on Demand IT InfraStructure (OASIS), 2004.
- [43] Willem de Bruijn and Herbert Bos. Beltway buffers: Avoiding the OS traffic jam. In INFOCOM, 2008.
 - [44] Anton Burtsev, Kiran Srinivasan, Prashanth Radhakrishnan, Kaladhar Voruganti, and Garth R. Goudon. Fido: Fast inter-virtual-machine communication for enterprise appliances. In 2009 USENIX Annual Technical Conference (ATC), June 2009.
 - [45] Andrew Ferraiuolo, Andrew Baumann, Chris Hawblitzel, and Bryan Parno. Komodo: Using verification to disentangle secure-enclave hardware from software. In Proceedings of the 26th Symposium on Operating Systems Principles (SOSP), 2017.
 - [46] John Rushby. Noninterference, transitivity, and channel-control security policies. 1992.
 - [47] Joseph A. Goguen and Jose Meseguer. Unwinding and inference control. In 1984 IEEE Symposium on Security and Privacy, 1984.
 - [48] Luke Nelson, James Bornholt, Arvind Krishnamurthy, Emina Torlak, and Xi Wang. Noninterference specifications for secure systems. SIGOPS Operating Systems Review, 1, August 2020.
 - [49] Robert Ricci, Eric Eide, and The CloudLab Team. Introducing CloudLab: Scientific infrastructure for advancing cloud architectures and applications. USENIX ;login:, 6, December 2014.
 - [50] CloudLab hardware info. URL: <https://docs.cloudlab.us/hardware.html>.
 - [51] Shih-Wei Li, Xupeng Li, Ronghui Gu, Jason Nieh, and John Zhuang Hui. Formally verified memory protection for a commodity multiprocessor hypervisor. In Proceedings of the 30th USENIX Security Symposium, 2021.
 - [52] Recursive proof in Verified NrOS page table. URL: https://github.com/matthias-brun/verified-nrkernel/blob/8f30cf0f910e7606c3a0f633821acfdfe410cf4/page-table/impl_u/l2_impl.rs#L947-L1562.
 - [53] DPDK: Data Plane Development Kit. URL: <https://www.dpdk.org>.
 - [54] Intel Corporation. Storage Performance Development Kit (SPDK). URL: <https://spdk.io>.
 - [55] Daniel E. Eisenbud, Cheng Yi, Carlo Contavalli, Cody Smith, Roman Kononov, Eric Mann-Hielscher, Ardas Cilingiroglu, Bin Cheyney, Wentao Shang, and Jinnah Dylan Hosein. Maglev: A fast and reliable software network load balancer. In Proceedings of the 13th USENIX Symposium on Networked Systems Design and Implementation (NSDI), March 2016.
 - [56] Rajesh Nishtala, Hans Fugal, Steven Grimm, Marc Kwiatkowski, Herman Lee, Harry C. Li, Ryan McElroy, Mike Paleczny, Daniel Peek, Paul Saab, David Stafford, Tony Tung, and Venkateshwa

- ran Venkataramani. Scaling memcache at Facebook. In Proceedings of the 10th USENIX Symposium on Networked Systems Design and Implementation (NSDI), April 2013.
- [57] Giuseppe DeCandia, Deniz Hastorun, Madan Jampani, Gunavardhan Kakulapati, Avinash Lakshman, Alex Pilchin, Swaminathan Sivasubramanian, Peter Voshall, and Werner Vogels. Dynamo: Amazon’s highly available key-value store. In Proceedings of the 21st ACM SIGOPS Symposium on Operating Systems Principles (SOSP), 2007.
- [58] NGINX. NGINX: High performance load balancer, web server, and reverse proxy. URL: <https://www.nginx.com>.
- [59] wrk - a HTTP benchmarking tool. URL: <https://github.com/wg/wrk>.
- [60] Department of defense trusted computer system evaluation criteria. In The “Orange Book” Series, 1985.
- [61] B. D. Gold, R. R. Linde, and P. F. Cudney. KVM/370 in retrospect. In 1984 IEEE Symposium on Security and Privacy, 1984.
- [62] P. A. Karger, M. E. Zurko, D. W. Bonin, A. H. Mason, and C. E. Kahn. A retrospective on the VAX VMM security kernel. IEEE Transactions on Software Engineering, 11, 1991.
- [63] L. J. Fraim. Scomp: A solution to the multilevel security problem. Computer, 7, July 1983.
- [64] W. R. Schockley, T. F. Tao, and M. F. Thompson. An overview of the GEMSOS Class A1 technology and application experience. In Proceedings of the 11th National Computer Security Conference, October 1988.
- [65] Fengwei Xu, Ming Fu, Xinyu Feng, Xiaoran Zhang, Hui Zhang, and Zhaohui Li. A practical verification framework for preemptive OS kernels. In Computer Aided Verification, 2016.
- [66] The UNSAT group. Hyperkernel git repository. URL: <https://github.com/uw-unsat/hyperkernel>.
- [67] Ankit Bhardwaj, Chinmay Kulkarni, Reto Achermann, Irina Calciu, Sanidhya Kashyap, Ryan Stutsman, Amy Tai, and Gerd Zellweger. NrOS: Effective replication and sharing in an operating system. In 15th USENIX Symposium on Operating Systems Design and Implementation (OSDI), July 2021.